

MCCC013-23

OTIMIZAÇÃO LINEAR

Aula 02 – I. Introdução

Saul C. Leite

Universidade Federal do ABC

16 de setembro de 2026

Conteúdo

1 Aula Passada

2 Ferramentas Computacionais

- Softwares de Otimização
- GLPK e a linguagem MathProg

3 Modelagem

- Exemplo 1: Fábrica de Fósforos
- Exemplo 2: Problema da Dieta
- Exemplo 3: Problema de Produção
- Exemplo 4: Orçamento de Capital
- Exemplo 5: Problema de Transporte
- Exemplo 6: Mistura de Carvão
- Exemplo 7: Agendamento de Produção

Conteúdo

1 Aula Passada

2 Ferramentas Computacionais

- Softwares de Otimização
- GLPK e a linguagem MathProg

3 Modelagem

- Exemplo 1: Fábrica de Fósforos
- Exemplo 2: Problema da Dieta
- Exemplo 3: Problema de Produção
- Exemplo 4: Orçamento de Capital
- Exemplo 5: Problema de Transporte
- Exemplo 6: Mistura de Carvão
- Exemplo 7: Agendamento de Produção

Problema de Programação Linear

Vimos na aula passada que problemas de **Programação Linear** (PPL) são problemas de otimização cuja função objetivo e restrições são **lineares**. O problema abaixo é um exemplo de PPL:

$$(P) \begin{cases} \min & c_1x_1 + \cdots + c_nx_n \\ \text{s.a.:} & a_{i1}x_1 + \cdots + a_{in}x_n \begin{pmatrix} = \\ \leq \\ \leq \\ \geq \end{pmatrix} b_i, \quad i = 1, \dots, m \\ & x_1 \geq 0, \dots, x_n \geq 0 \end{cases}$$

Terminologia

Lembre-se que:

- $c_1x_1 + \dots + c_nx_n$ é a **função objetivo**, em que $c_1, \dots, c_n$ são constantes conhecidas;
- $x_1, \dots, x_n$ são as **variáveis de decisão** (temos n variáveis);
- cada linha $a_{i1}x_1 + \dots + a_{in}x_n$ ($=, \leq$ ou $\geq$) b_i é uma **restrição**, em que a_{ij} e b_i são constantes conhecidas. Temos m restrições.

Notação Matricial

É comum escrever o problema em **notação matricial**:

$$(P) \begin{cases} \min & c^\top x \\ \text{s.a.:} & Ax = b \\ & x \geq 0 \end{cases} \quad \begin{array}{l} \circ \ x, c \in \mathbb{R}^n; \\ \circ \ b \in \mathbb{R}^m; \\ \circ \ A \in \mathbb{R}^{m \times n}, \end{array}$$

em que

$$A = \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix},$$

supondo que todas as restrições são de igualdade.

Conteúdo

1 Aula Passada

2 Ferramentas Computacionais

- Softwares de Otimização
- GLPK e a linguagem MathProg

3 Modelagem

- Exemplo 1: Fábrica de Fósforos
- Exemplo 2: Problema da Dieta
- Exemplo 3: Problema de Produção
- Exemplo 4: Orçamento de Capital
- Exemplo 5: Problema de Transporte
- Exemplo 6: Mistura de Carvão
- Exemplo 7: Agendamento de Produção

Softwares de Otimização

Resolvendo os Modelos

Na aula anterior, vimos como representar um problema prático por meio de um modelo de Programação Linear (PL).

Resolvendo os Modelos

Na aula anterior, vimos como representar um problema prático por meio de um modelo de **Programação Linear** (PL).

O próximo passo é **resolver** o modelo:

- existem diversos pacotes computacionais dedicados a isso;
- basta descrever o modelo em uma linguagem adequada e o pacote devolve a solução ótima.

Softwares Comerciais

Gurobi: comercializado pela Gurobi Optimization Inc., é hoje um dos solvers mais rápidos para PL e programação inteira. (*licença acadêmica gratuita*)

Cplex: chamado de “IBM ILOG CPLEX Optimization Studio”, é um dos mais tradicionais pacotes comerciais para resolução de problemas de PL. (*licença acadêmica gratuita*)

Xpress: o Xpress Optimization Suite é desenvolvido pela empresa FICO, especializada em software para análise de dados.

Outros: **MOSEK** (forte em pontos interiores) e **COPT**, ambos também com licença acadêmica gratuita.

Softwares Livres

GLPK: *GNU Linear Programming Kit*. Implementa o simplex primal, o dual e um algoritmo de pontos interiores. (é o que usaremos nesta disciplina)

HiGHS: solver livre (licença MIT) de alto desempenho para PL, desenvolvido na Universidade de Edimburgo. É o solver usado pelo `scipy.optimize.linprog`, em Python, e está disponível também em pacotes de outras linguagens.

CLP: parte do projeto *Computational Infrastructure for Operations Research* – Coin-OR (<https://www.coin-or.org>), que reúne várias ferramentas de pesquisa operacional.

SCIP: distribuído sob licença Apache 2.0; voltado principalmente a problemas de programação inteira.

GLPK e a linguagem MathProg

GNU Linear Programming Kit

O GLPK será útil para testarmos nossos modelos.

1. Livre e parte do projeto GNU;
2. implementa a linguagem GNU MathProg, que pode ser convertida para a entrada de outros pacotes usando o próprio GLPK:
 - formato CPLEX LP;
 - formato MPS, usado pela maioria dos pacotes;
3. a **linguagem MathProg** é um subconjunto da linguagem AMPL (*A Mathematical Programming Language*), geralmente usada com os softwares comerciais listados acima;
4. funciona no Windows, Linux e Mac.

Conteúdo

1 Aula Passada

2 Ferramentas Computacionais

- Softwares de Otimização
- GLPK e a linguagem MathProg

3 Modelagem

- Exemplo 1: Fábrica de Fósforos
- Exemplo 2: Problema da Dieta
- Exemplo 3: Problema de Produção
- Exemplo 4: Orçamento de Capital
- Exemplo 5: Problema de Transporte
- Exemplo 6: Mistura de Carvão
- Exemplo 7: Agendamento de Produção

Exemplo 1: Fábrica de Fósforos

Exemplo 1: Fábrica de Fósforos (Problema do

Prof. Celso Ribeiro)

Uma empresa produz dois tipos de fósforos: **longos** e **curtos**.

- lucro: \$3 por 100 caixas de fósforos longos e \$2 por 100 caixas de fósforos curtos;
- os dois tipos são produzidos na mesma máquina, que produz até 900.000 caixas por ano;
- madeira por 100.000 caixas: 3 m^3 (longos) e 1 m^3 (curtos);
- total de madeira disponível: 18 m^3 ;
- embalagens disponíveis: 700.000 (longos) e 600.000 (curtos).

Queremos **maximizar o lucro**, supondo que toda a produção seja vendida.

Exemplo 1: Modelo (Aula 01)

Com x_1 e x_2 denotando a produção de fósforos longos e curtos (em 100.000 caixas), obtivemos:

$$\begin{array}{llll} \max & 3x_1 & + & 2x_2 \\ \text{s.a.:} & x_1 & + & x_2 \leq 9 \quad (\text{máquina}) \\ & 3x_1 & + & x_2 \leq 18 \quad (\text{madeira}) \\ & x_1 & & \leq 7 \quad (\text{embalagens longas}) \\ & & & x_2 \leq 6 \quad (\text{embalagens curtas}) \\ & x_1, x_2 & \geq & 0 \end{array}$$

Vejamos como escrever este modelo em **MathProg**.

Exemplo 1: Usando MathProg

```
var x1 >=0; # fosforos longos (x 100 mil)
var x2 >=0; # fosforos curtos (x 100 mil)

maximize z: 3*x1 + 2*x2;

s.t. r1:    x1 + x2 <= 9;      # max. maquina
s.t. r2: 3*x1 + x2 <= 18;     # max. madeira
s.t. r3:    x1          <= 7;  # max. embal. longas
s.t. r4:          x2 <= 6;     # max. embal. curtas

solve;

display x1, x2, z;
```

Executando o Modelo (Linux/Mac)

No Linux ou Mac, após instalar o GLPK, resolvemos o problema com a seguinte linha de comando:

```
$glpsol -m arq.glpk
```

em que arq.glpk é o nome do arquivo com o texto do slide anterior. A saída é a seguinte:

```
OPTIMAL LP SOLUTION FOUND
Time used:    0.0 secs
Memory used:  0.1 Mb (102266 bytes)
Display statement at line 13
x1.val = 4.5
x2.val = 4.5
z.val = 22.5
Model has been successfully processed
```

Exemplo 1: Solução

Ou seja, a **solução ótima** é

$$x_1^* = 4,5, \quad x_2^* = 4,5, \quad z^* = 22,5,$$

que corresponde a produzir:

- 450.000 caixas de fósforos longos;
- 450.000 caixas de fósforos curtos,

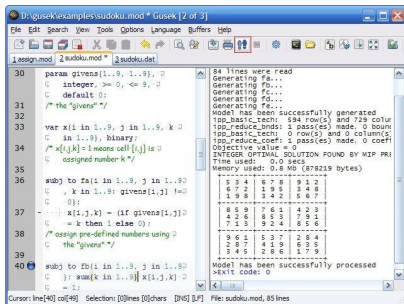
com lucro de \$22.500 – como havíamos obtido na Aula 01.

Executando o Modelo (Windows)

No Windows, o GLPK possui a interface **GUSEK**

<http://gusek.sourceforge.net/gusek.html>.

Com o GUSEK aberto, abra o arquivo do modelo em File → Open.



GLPK-ONLINE

Podemos também usar uma interface disponível no seguinte endereço:

<https://cocoto.github.io/glpk-online/>

As variáveis impressas aparecem na aba “output”.

Exemplo 2: Problema da Dieta

Exemplo 2: Problema da Dieta (Problema do

Prof. Celso Ribeiro)

Suponha que uma dieta alimentar está restrita a:

- leite desnatado (\$2 o litro);
- carne (\$4 o quilo);
- peixe (\$1,5 o quilo);
- salada (\$2 por 100 g).

Os requisitos nutricionais mínimos recomendados são expressos em miligramas das vitaminas A , C e D .

Gostaríamos de determinar a **dieta mais barata** que satisfaz as necessidades nutricionais mínimas.

Exemplo 2: Dados

Vitamina	leite	carne	peixe	salada	Req. Mín.
A	2 mg/l	2 mg/kg	90 mg/kg	20 mg/100g	11 mg
C	50 mg/l	20 mg/kg	30 mg/kg	30 mg/100g	70 mg
D	80 mg/l	70 mg/kg	10 mg/kg	60 mg/100g	250 mg

A última coluna traz a quantidade mínima de cada vitamina que a dieta deve conter.

Exemplo 2: Modelo

1) Variáveis de decisão:

- x_1 : quantidade de leite (litros);
- x_2 : quantidade de carne (kg);
- x_3 : quantidade de peixe (kg);
- x_4 : quantidade de salada (porções de 100 g).

Exemplo 2: Modelo

1) Variáveis de decisão:

- x_1 : quantidade de leite (litros);
- x_2 : quantidade de carne (kg);
- x_3 : quantidade de peixe (kg);
- x_4 : quantidade de salada (porções de 100 g).

2) Objetivo:

$$\min \text{ custo} = 2x_1 + 4x_2 + 1,5x_3 + 2x_4 \quad (\text{em \$}).$$

Exemplo 2: Modelo

3) Restrições:

$$\begin{array}{ll} 2x_1 + 2x_2 + 90x_3 + 20x_4 \geq 11 & \text{(vitamina A)} \\ 50x_1 + 20x_2 + 30x_3 + 30x_4 \geq 70 & \text{(vitamina C)} \\ 80x_1 + 70x_2 + 10x_3 + 60x_4 \geq 250 & \text{(vitamina D)} \end{array}$$

$$x_1, x_2, x_3, x_4 \geq 0 \quad \text{(não negatividade).}$$

Exemplo 2: Problema de PL

$$\begin{array}{llllll} \min & 2x_1 & + & 4x_2 & + & 1,5x_3 & + & 2x_4 \\ \text{s.a.:} & 2x_1 & + & 2x_2 & + & 90x_3 & + & 20x_4 & \geq & 11 \\ & 50x_1 & + & 20x_2 & + & 30x_3 & + & 30x_4 & \geq & 70 \\ & 80x_1 & + & 70x_2 & + & 10x_3 & + & 60x_4 & \geq & 250 \\ & x_1, x_2, x_3, x_4 & \geq & 0 \end{array}$$

Em notação matricial: $\min c^\top x$ s.a. $Ax \geq b, \quad x \geq 0$, com

$$c = \begin{bmatrix} 2 \\ 4 \\ 1,5 \\ 2 \end{bmatrix}, \quad A = \begin{bmatrix} 2 & 2 & 90 & 20 \\ 50 & 20 & 30 & 30 \\ 80 & 70 & 10 & 60 \end{bmatrix}, \quad b = \begin{bmatrix} 11 \\ 70 \\ 250 \end{bmatrix}.$$

Exemplo 2: Usando MathProg

```
var x1 >= 0; # litros de leite
var x2 >= 0; # quilo de carne
var x3 >= 0; # quilo de peixe
var x4 >= 0; # 100g de salada

minimize z: 2*x1 + 4*x2 + 1.5*x3 + 2*x4;

s.t. vita_a: 2*x1 + 2*x2 + 90*x3 + 20*x4 >= 11;
s.t. vita_c: 50*x1 + 20*x2 + 30*x3 + 30*x4 >= 70;
s.t. vita_d: 80*x1 + 70*x2 + 10*x3 + 60*x4 >= 250;

solve;

display x1, x2, x3, x4, z;
```

Exemplo 2: Saída do GLPK

A saída do glpsol é:

```
OPTIMAL LP SOLUTION FOUND
Time used:    0.0 secs
Memory used:  0.1 Mb (102229 bytes)
Display statement at line 15
x1.val = 3.11838440111421
x2.val = 0
x3.val = 0.052924791086351
x4.val = 0
z.val = 6.31615598885794
Model has been successfully processed
```

Exemplo 2: Solução

Arredondando, a **solução ótima** é

$$x^* = \begin{bmatrix} 3,12 \\ 0 \\ 0,053 \\ 0 \end{bmatrix}, \quad z^* = 6,32.$$

Ou seja, a dieta mais barata consiste em:

- 3,12 litros de leite;
- 53 gramas de peixe,

com custo de \$6,32.

Nota Histórica

O problema da dieta tem um papel especial na história da PL:

- o primeiro problema de PL de “larga escala” resolvido pelo método simplex, em 1947, foi um problema de dieta (proposto por G. Stigler);
- o problema tinha 77 variáveis e 9 restrições;
- foi resolvido manualmente, com calculadoras de mesa, e levou cerca de 120 homens-dia de trabalho.

Veja um exemplo similar na Seção 2.1 (p. 12) de Matoušek & Gärtner.

Exemplo 3: Problema de Produção

Exemplo 3: Problema de Produção

(Simplificação de Bertsimas & Tsitsiklis, pg. 7)

Este exemplo é baseado em um problema real da *Digital Equipment Corporation* (DEC), em 1988. A empresa produz:

- *general purpose systems*: GP-1, GP-2, GP-3;
- *work stations*: WS-1, WS-2.

Sistema	Preço	No. Un. Disco	No. Placa de Memória 256K
GP-1	\$60.000	0,3*	4
GP-2	\$40.000	1,7	2
GP-3	\$30.000	0	2
WS-1	\$30.000	1,4	2
WS-2	\$15.000	0	1

(*) 3 a cada 10 clientes pediram uma unidade de disco (no trimestre anterior).

Exemplo 3: Fornecimento

As seguintes dificuldades são antecipadas para o próximo trimestre:

(a) Fornecimento:

1. o fornecedor de CPU produz no máximo 7 mil unidades;
2. o fornecimento de unidades de disco é estimado entre 3 mil e 7 mil unidades;
3. o fornecimento de placas de memória é estimado entre 8 mil e 16 mil unidades.

Exemplo 3: Demanda

(b) Demanda:

1. projeções: 1800 unidades de GP-1, 300 de GP-3, 3800 da família GP e 3200 da família WS;
2. pedidos: 500 para GP-2, 500 para WS-1 e 400 para WS-2 (a demanda acima já inclui esses pedidos).

Exemplo 3: Demanda

(b) Demanda:

1. projeções: 1800 unidades de GP-1, 300 de GP-3, 3800 da família GP e 3200 da família WS;
2. pedidos: 500 para GP-2, 500 para WS-1 e 400 para WS-2 (a demanda acima já inclui esses pedidos).

Quantas unidades de cada computador devemos produzir para ter a maior arrecadação e atender os pedidos?

Exemplo 3: Modelo

1) **Variáveis de decisão** (em mil unidades):

- x_1, x_2, x_3 : produção de GP-1, GP-2 e GP-3;
- x_4, x_5 : produção de WS-1 e WS-2.

Exemplo 3: Modelo

1) Variáveis de decisão (em mil unidades):

- x_1, x_2, x_3 : produção de GP-1, GP-2 e GP-3;
- x_4, x_5 : produção de WS-1 e WS-2.

2) Objetivo:

$$\max z = \$60.000 (1.000 x_1) + \cdots + \$15.000 (1.000 x_5),$$

ou seja,

$$\max z = 60x_1 + 40x_2 + 30x_3 + 30x_4 + 15x_5$$

(em milhões de \$).

Exemplo 3: Restrições

3) Restrições:

(a) Fornecimento:

$$x_1 + x_2 + x_3 + x_4 + x_5 \leq 7 \quad (\text{CPU})$$

$$0,3x_1 + 1,7x_2 + 1,4x_4 \leq 3 \quad (\text{disco})$$

$$4x_1 + 2x_2 + 2x_3 + 2x_4 + x_5 \leq 8 \quad (\text{memória})$$

Como o fornecimento de discos e de placas de memória é incerto, usamos as **estimativas mais pessimistas** (3 mil e 8 mil unidades).

Exemplo 3: Restrições (cont.)

(b) Demanda:

$x_1 \leq 1,8$	(projeção GP-1)
$x_3 \leq 0,3$	(projeção GP-3)
$x_1 + x_2 + x_3 \leq 3,8$	(projeção família GP)
$x_4 + x_5 \leq 3,2$	(projeção família WS)
$x_2 \geq 0,5$	(pedidos GP-2)
$x_4 \geq 0,5$	(pedidos WS-1)
$x_5 \geq 0,4$	(pedidos WS-2)

(c) Não negatividade: $x_1, \dots, x_5 \geq 0$.

Exemplo 3: Usando MathProg

```
var x1 >=0;
var x2 >=0;
var x3 >=0;
var x4 >=0;
var x5 >=0;

maximize z: 60*x1 + 40*x2 +30*x3 + 30*x4 + 15*x5;

s.t. a:      x1 +      x2 + x3  + x4      + x5 <= 7;
s.t. b: 0.3*x1 + 1.7*x2      + 1.4*x4      <= 3;
s.t. c:  4*x1 +   2*x2 +2*x3 + 2*x4      + x5 <= 8;
s.t. d:      x1                                     <=1.8;
s.t. e:                                     x3      <=0.3;
s.t. f:      x1 +      x2 +  x3                                     <=3.8;
s.t. g:                                     x4      + x5 <=3.2;
s.t. h:      x2                                     >=0.5;
s.t. i:                                     x4      >=0.5;
s.t. j:                                     x5 >=0.4;

solve;

display x1, x2, x3, x4, x5, z;
```

Exemplo 3: Solução

A **solução ótima** fornecida pelo GLPK é:

Sistema	Variável	Valor ótimo	Unidades
GP-1	x_1	0,2726	273
GP-2	x_2	1,3048	1.305
GP-3	x_3	0,3	300
WS-1	x_4	0,5	500
WS-2	x_5	2,7	2.700

com $z^* \approx 133,05$, ou seja, uma arrecadação de cerca de **\$133 milhões** (unidades arredondadas).

Exemplo 4: Orçamento de Capital

Exemplo 4: Orçamento de Capital

(adaptado de Bazaraa, Jarvis & Sherali, 3ª ed., Seção 1.2)

Uma cidade possui um projeto de construção com previsão de duração de 4 anos. A prefeitura venderá títulos de longo prazo para arrecadar dinheiro para a obra.

Custos da obra:

- \$2 milhões – ano 1;
- \$4 milhões – ano 2;
- \$8 milhões – ano 3;
- \$5 milhões – ano 4.

Juros dos títulos:

- 7% – ano 1;
- 6% – ano 2;
- 6,5% – ano 3;
- 7,5% – ano 4.

Todo o dinheiro de um ano é necessário no **início** do ano.

Exemplo 4: Orçamento de Capital (cont.)

Cada título tem **prazo de 20 anos**: a prefeitura paga juros anualmente, a partir do ano seguinte à venda, e devolve o valor ao final do prazo. Os juros são pagos com **outros recursos do orçamento**, e não com o dinheiro da obra.

A prefeitura pode colocar dinheiro em fundos de curta duração (1 ano) com rendimento de 6% (ano 1), 5,5% (ano 2) e 4,5% (ano 3). No quarto ano, não há investimento de curta duração.

Todos os valores estão em **moeda de hoje**, e as taxas são reais (já descontada a inflação).

Qual a estratégia que a prefeitura deverá adotar para minimizar o total pago em juros aos detentores dos títulos?

Exemplo 4: Modelo

1) **Variáveis de decisão** (em milhões de \$):

- x_i : dinheiro arrecadado com títulos no ano i ,
 $i = 1, \dots, 4$;
- y_i : quantia investida em fundos de curta duração no ano i , $i = 1, 2, 3$.

Exemplo 4: Modelo

1) **Variáveis de decisão** (em milhões de \$):

- x_i : dinheiro arrecadado com títulos no ano i ,
 $i = 1, \dots, 4$;
- y_i : quantia investida em fundos de curta duração no ano i , $i = 1, 2, 3$.

2) **Objetivo** (total pago em juros por 20 anos):

$$\min 20(0,07)x_1 + 20(0,06)x_2 + 20(0,065)x_3 + 20(0,075)x_4$$

Exemplo 4: Restrições

3) Restrições (balanço de caixa em cada ano):

$$x_1 - y_1 = 2 \quad (\text{ano 1})$$

$$1,06 y_1 + x_2 - y_2 = 4 \quad (\text{ano 2})$$

$$1,055 y_2 + x_3 - y_3 = 8 \quad (\text{ano 3})$$

$$1,045 y_3 + x_4 = 5 \quad (\text{ano 4})$$

$$x_1, x_2, x_3, x_4, y_1, y_2, y_3 \geq 0.$$

Em cada ano: (resgate do fundo anterior) + (títulos) – (novo investimento) = (custo da obra).

Poderíamos usar $\geq$, mas qualquer excesso seria depositado; por isso a igualdade é adequada.

Exemplo 4: Usando MathProg

```
var x1 >= 0;
var x2 >= 0;
var x3 >= 0;
var x4 >= 0;
var y1 >= 0;
var y2 >= 0;
var y3 >= 0;

minimize z: 20*(0.07)*x1 + 20*(0.06)*x2
           + 20*(0.065)*x3 + 20*(0.075)*x4;

s.t. a: x1 - y1 = 2;
s.t. b: x2 + (1.060)*y1 - y2 = 4;
s.t. c: x3 + (1.055)*y2 - y3 = 8;
s.t. d: x4 + (1.045)*y3 = 5;

solve;

display x1, x2, x3, x4, y1, y2, y3, z;
```

Exemplo 4: Solução

A **solução ótima** (em milhões de \$) é:

$$x_1^* = 2$$

$$y_1^* = 0$$

$$x_2^* = 16,1182$$

$$y_2^* = 12,1182$$

$$x_3^* = x_4^* = 0$$

$$y_3^* = 4,7847$$

Ou seja, a prefeitura arrecada tudo nos anos 1 e 2 (o ano 2 tem os menores juros, 6%) e aplica o excedente em fundos para cobrir os anos 3 e 4. O total pago em juros é $z^* \approx \$22,14$ milhões.

Exemplo 5: Problema de Transporte

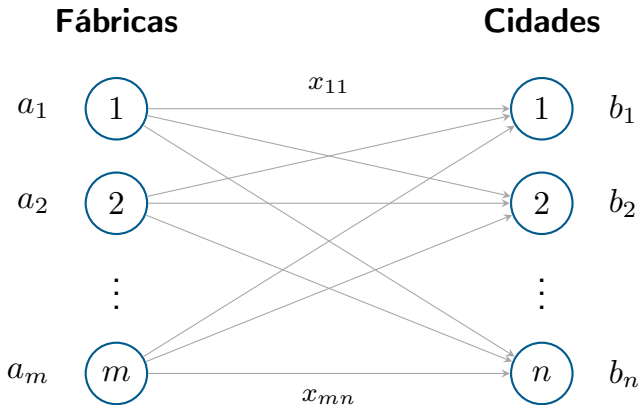
Exemplo 5: Problema de Transporte

Uma empresa produz um determinado produto X em m fábricas diferentes. A capacidade de produção de cada fábrica $i = 1, \dots, m$ é de a_i .

Os produtos são consumidos em n cidades diferentes, afastadas das fábricas. Cada cidade $j = 1, \dots, n$ tem demanda de b_j unidades do produto.

Sabendo que o custo de envio de uma unidade do produto da fábrica i para a cidade j é de c_{ij} , determine a quantidade de produto que deve ser enviada de cada fábrica para cada cidade de modo a **minimizar os custos de transporte**.

Exemplo 5: Representação em Rede



1) Variáveis de decisão: x_{ij} : quantidade transportada da fábrica i para a cidade j .

Modelo geral e figura: Bazaraa, Jarvis & Sherali, 3ª ed., Seção 1.2 (Fig. 1.2).

Exemplo 5: Modelo Geral

2) Objetivo:

$$\min \text{ custo} = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

Exemplo 5: Modelo Geral

2) Objetivo:

$$\min \text{ custo} = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

3) Restrições:

$$\sum_{j=1}^n x_{ij} \leq a_i, \quad i = 1, \dots, m \quad (\text{capacidade})$$

$$\sum_{i=1}^m x_{ij} = b_j, \quad j = 1, \dots, n \quad (\text{demanda})$$

$$x_{ij} \geq 0, \quad i = 1, \dots, m; \quad j = 1, \dots, n.$$

Exemplo 5: Caso Particular

Considere um caso particular em que:

$m = 2$, fábricas F1 e F2;

$n = 3$, cidades C1, C2 e C3,

com os seguintes dados:

$$a = \begin{matrix} F_1 \\ F_2 \end{matrix} \begin{bmatrix} 250 \\ 235 \end{bmatrix} \quad b = \begin{matrix} C_1 \\ C_2 \\ C_3 \end{matrix} \begin{bmatrix} 75 \\ 240 \\ 70 \end{bmatrix} \quad c = \begin{matrix} & C_1 & C_2 & C_3 \\ F_1 & 15 & 16 & 21 \\ F_2 & 15 & 7 & 17 \end{matrix}$$

Exemplo 5: Modelo do Caso Particular

Nesse caso, o problema toma a seguinte forma:

$$(P) \left\{ \begin{array}{ll} \min & 15x_{11} + 16x_{12} + 21x_{13} + 15x_{21} + 7x_{22} + 17x_{23} \\ \text{s.a.} & x_{11} + x_{12} + x_{13} \leq 250 \\ & x_{21} + x_{22} + x_{23} \leq 235 \\ & x_{11} + x_{21} = 75 \\ & x_{12} + x_{22} = 240 \\ & x_{13} + x_{23} = 70 \\ & x_{ij} \geq 0, \quad i = 1, 2; \quad j = 1, 2, 3. \end{array} \right.$$

Exemplo 5: Usando MathProg

```
# Conjuntos
set F;
set C;

# Parametros
param c {i in F, j in C};
param a {i in F};
param b {j in C};

# Variaveis
var x {i in F, j in C} >= 0;

# Objetivo
minimize z: sum{ i in F, j in C} x[i,j] * c[i,j];

# Restricoes
s.t. producao { i in F }: sum{ j in C } x[i,j] <= a[i];
s.t. demanda { j in C }: sum{ i in F } x[i,j] = b[j];

solve;
printf{ i in F, j in C}: 'x[%s,%s] = %f\n',i,j,x[i,j];
display z;
```

Exemplo 5: Usando MathProg (cont.)

No mesmo arquivo, podemos incluir uma seção de dados para o problema. No caso acima, teremos:

```
data;
```

```
set F := F1 F2;
```

```
set C := C1 C2 C3;
```

```
param c : C1 C2 C3 :=
```

```
    F1 15 16 21
```

```
    F2 15  7 17;
```

```
param a :=
```

```
    F1 250
```

```
    F2 235;
```

```
param b :=
```

```
    C1 75
```

```
    C2 240
```

```
    C3 70;
```

```
end;
```

Exemplo 5: Usando MathProg (cont.)

A seguinte expressão pode ser usada antes da seção de dados para fazer a impressão do resultado:

```
printf{ i in F, j in C}: 'x[%s,%s] = %f\n',i,j,x[i,j];  
display z;
```

Para maiores detalhes, veja o livro da linguagem AMPL, disponível gratuitamente para download:

<https://ampl.com/learn/ampl-book/>

Exemplo 5: Solução

A **solução ótima** do caso particular é:

	C1	C2	C3
F1	$x_{11}^* = 75$	$x_{12}^* = 5$	$x_{13}^* = 70$
F2	$x_{21}^* = 0$	$x_{22}^* = 235$	$x_{23}^* = 0$

O **custo mínimo** é de \$4.320.

A fábrica F2 usa toda a sua capacidade (235) e abastece apenas C2, a rota mais barata ($c_{22} = 7$). F1 envia 150 das 250 unidades disponíveis.

Exemplo 6: Mistura de Carvão

Exemplo 6: Mistura de Carvão

A *Wyoming Electric Coop* tem uma usina de turbinas a vapor. Como o estado de Wyoming é rico em depósitos de carvão, a usina utiliza carvão para gerar vapor. No entanto, isso pode gerar emissões que não cumprem os padrões da EPA.

As regulamentações limitam a descarga de dióxido de enxofre a 2.000 partes por milhão *por tonelada de carvão* e a descarga de fumaça pelas chaminés a 20 lb por hora. A companhia recebe dois tipos de carvão, C1 e C2. Considere que a descarga de enxofre é uma média ponderada entre as proporções de cada categoria utilizada.

A tabela a seguir traz os dados do consumo de 1 tonelada de carvão por hora. Determine quanto de cada carvão a usina deve queimar por hora para **maximizar o vapor gerado**, respeitando os limites de emissão.

Exemplo 6: Dados

Categoria do carvão	Descarga de enxofre (partes por milhão)	Descarga de fumaça (lb por hora)	Vapor gerado (lb por hora)
<i>C1</i>	1.800	2,1	12.000
<i>C2</i>	2.100	0,9	9.000

Note que o carvão C1 possui menor descarga de enxofre e produz mais vapor, mas tem maior descarga de fumaça. O carvão C2 possui maior descarga de enxofre, mas menor descarga de fumaça e produz menos vapor.

Exemplo 6: Modelo

1) Variáveis de decisão:

- x_1 : toneladas de carvão C1 queimadas por hora;
- x_2 : toneladas de carvão C2 queimadas por hora.

Exemplo 6: Modelo

1) Variáveis de decisão:

- x_1 : toneladas de carvão C1 queimadas por hora;
- x_2 : toneladas de carvão C2 queimadas por hora.

2) Objetivo (vapor gerado, em lb por hora):

$$\max z = 12000 x_1 + 9000 x_2$$

Exemplo 6: Restrições

3) Restrições:

Fumaça (no máximo 20 lb por hora):

$$2,1 x_1 + 0,9 x_2 \leq 20$$

Exemplo 6: Restrições

3) Restrições:

Fumaça (no máximo 20 lb por hora):

$$2,1 x_1 + 0,9 x_2 \leq 20$$

Enxofre (média ponderada de no máximo 2.000 ppm):

$$\frac{1800 x_1 + 2100 x_2}{x_1 + x_2} \leq 2000 \implies -200 x_1 + 100 x_2 \leq 0$$

Obs.: Multiplicamos os dois lados por $x_1 + x_2 \geq 0$, o que não altera o sentido da desigualdade.

Não negatividade: $x_1, x_2 \geq 0$.

Exemplo 6: Usando MathProg

```
#Variaveis
var x1 >= 0;
var x2 >= 0;

#Funcao Objetivo
maximize z: 12000*x1 + 9000*x2;

#Restricoes
s.t. r1: -200*x1 + 100*x2 <= 0;
s.t. r2: 2.1*x1 + 0.9*x2 <= 20;

solve;

display z,x1,x2;
```

Exemplo 6: Solução

A **solução ótima** é

$$x_1^* \approx 5,13, \quad x_2^* \approx 10,26, \quad z^* \approx 153\,846,$$

ou seja, a usina deve queimar cerca de:

- 5,13 toneladas de C1 por hora;
- 10,26 toneladas de C2 por hora,

gerando cerca de 153 846 lb de vapor por hora.

A mistura tem proporção 1 : 2 (C1 : C2), com enxofre de exatamente 2.000 ppm e fumaça de exatamente 20 lb/h: as duas restrições estão ativas.

Exemplo 7: Agendamento de Produção

Exemplo 7: Agendamento de Produção

Um administrador está planejando o agendamento de 3 produtos em 4 máquinas. Cada um dos produtos pode ser fabricado em qualquer máquina.

- o custo de produção de cada produto em cada máquina é dado pela tabela (a) a seguir;
- o tempo, em horas, gasto para produzir uma unidade de cada produto em cada máquina é dado pela tabela (b);
- são necessárias 3000, 6000 e 4000 unidades de cada produto;
- o tempo disponível de máquina é de 1500, 1200, 1500 e 2000 horas.

Formule um problema de PL que nos ajude a fabricar todos os produtos da forma mais barata.

Exemplo 7: Dados

(a) custo de produção

Produtos	Máquinas			
	M1	M2	M3	M4
p_1	4	4	5	7
p_2	6	7	5	6
p_3	12	10	8	11

(b) tempo de produção (horas)

Produtos	Máquinas			
	M1	M2	M3	M4
p_1	0,3	0,25	0,2	0,2
p_2	0,2	0,3	0,2	0,25
p_3	0,8	0,6	0,6	0,5

Exemplo 7: Usando MathProg

```
#Conjuntos
set Prod;
set Maq;
#Parametros
param C { i in Prod, j in Maq };
param T { i in Prod, j in Maq };
param num {i in Prod};
param temp {j in Maq};

#Variaveis
var x {i in Prod, j in Maq} >= 0;

#Objetivo
minimize z: sum{j in Maq} sum{ i in Prod } C[i,j] * x[i,j];
#Restricoes
s.t. q{i in Prod} : sum{j in Maq} x[i,j] = num[i];
s.t. t{j in Maq} : sum{i in Prod} T[i,j]*x[i,j] <= temp[j];
```

Exemplo 7: Seção de Dados

```
data;
set Prod := p1 p2 p3;
set Maq  := m1 m2 m3 m4;

param num :=
p1 3000
p2 6000
p3 4000;

param temp :=
m1 1500
m2 1200
m3 1500
m4 2000;

param C: m1 m2 m3 m4 :=
p1 4 4 5 7
p2 6 7 5 6
p3 12 10 8 11;

param T: m1 m2 m3 m4 :=
p1 0.30 0.25 0.20 0.20
p2 0.20 0.30 0.20 0.25
p3 0.80 0.60 0.60 0.50;

end;
```